

✓ How does Python manage memory?

This document explains how **CPython** (the reference implementation of Python, written in C) allocates, reuses and frees memory. Each section cites the source of its information in brackets (e.g. [F1]). The full list of sources is at the end.

```
import sys

print(sys.version) # several results (immutable objects, incremental GC) depend on the version

3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)]
```

0. Starting point: the internals guide

The [devguide's internals guide](#) [F1] is the official entry point to the documentation on "how CPython works inside". Since 2024, most of that content has moved to the `InternalDocs/` directory of the CPython repository, and the devguide links there. That is why the technical detail in this document comes mainly from `InternalDocs/garbage_collector.md` [F2] and `InternalDocs/string_interning.md` [F3], complemented by the official Python documentation [F4–F8] and the PEPs [F9–F11].

The big picture is that Python manages memory in **three layers**:

Layer	What it solves	Section
Allocator	Where does the memory for a new object come from?	§2
Reference counting	When is an object no longer used? (the normal case)	§3
Cyclic garbage collector (GC)	What about objects that reference each other in a cycle?	§4

✓ 1. Everything is an object, and variables are labels

In Python **everything** (integers, lists, functions, classes, modules) is an object that lives in Python's **private heap**. The programmer never allocates or frees memory directly: the interpreter does it. [F4]

"Memory management in Python involves a private heap containing all Python objects and data structures. The management of this private heap is ensured internally by the Python memory manager." — [F4]

A variable **does not contain** the object: it is a **name that points to** (references) an object.

```
a = [1, 2, 3]
b = a # b is NOT a copy: it points to the same object
b.append(4)
print(a) # [1, 2, 3, 4]
print(a is b) # True -> same object in memory
print(id(a), id(b)) # same id (memory address)
```

```
[1, 2, 3, 4]
True
2554753592896 2554753592896
```

In C, every object starts with a common header (`PyObject`) that stores, among other things:

- `ob_refcnt`: **how many references** point to the object.
- `ob_type`: a pointer to the object's **type** (`int`, `list`, ...).

[F2] [F5]

2. Where does memory come from? The `pymalloc` allocator

Asking the operating system for memory (`malloc`) for every small object would be slow, and Python programs create and destroy **huge numbers** of small, short-lived objects. That is why CPython uses its own allocator: **pymalloc**. [F5]

2.1 Hierarchy: arenas → pools → blocks

```
Operating system
├── Arena (1 MiB on 64-bit / 256 KiB on 32-bit)
│   └── Pool (holds blocks of ONE single size)
│       └── Block (where an object lives)
```

- **Arena**: a large chunk of memory requested from the OS with `mmap()` (or `VirtualAlloc()` on Windows). [F5]

- **Pool:** a subdivision of an arena; all its blocks have the same size ("size class").
- **Block:** the space where a single object is stored.

"pymalloc [...] is optimized for small objects (smaller or equal to 512 bytes) with a short lifetime. It uses memory mappings called 'arenas' with a fixed size of either 256 KiB on 32-bit platforms or 1 MiB on 64-bit platforms. It falls back to PyMem_RawMalloc() and PyMem_RawRealloc() for allocations larger than 512 bytes." — [F5]

Practical consequences:

- Objects ≤ 512 bytes $\rightarrow$ pymalloc (fast, reuses blocks).
- Objects > 512 bytes $\rightarrow$ the system's `malloc`.
- When an object is freed, its block goes back to the pool to be **reused**; the memory is not always returned to the OS right away (an arena is only released when it becomes completely empty). [F5]

2.2 Memory domains

The C API distinguishes three "domains": `PyMem_Raw*` (raw memory), `PyMem_*` (general memory) and `PyObject_*` (Python objects). The allocator can be changed with the `PYTHONMALLOC` environment variable (for example `PYTHONMALLOC=malloc` to disable pymalloc while debugging). [F5]

2.3 Free-threaded build (no GIL)

In the experimental build without the GIL (Python 3.13+), CPython uses **mimalloc** instead of pymalloc, because it needs an allocator that is safe for multiple threads. [F5] [F10]

3. The main mechanism: reference counting

3.1 The idea

Every object keeps count of how many references point to it (`ob_refcnt`). [F2]

- It is **incremented** when the object is assigned to a variable, put into a list, passed to a function, etc.
- It is **decremented** when a variable is reassigned, goes out of scope, is removed with `del`, etc.
- When it reaches 0, the object is destroyed **immediately** and its memory is freed.

"The main garbage collection algorithm used by CPython is reference counting. The basic idea is that CPython counts how many different places there are that have a reference to an object. [...] When an object's reference count becomes zero, the object is deallocated." — [F2]

3.2 Seeing it in action

```
import sys

x = []
print(sys.getrefcount(x)) # 2 (x + the temporary argument of getrefcount)

y = x
print(sys.getrefcount(x)) # 3

del y
print(sys.getrefcount(x)) # 2

2
3
2
```

`sys.getrefcount()` returns a value **one higher** than expected because the function's own argument is a temporary reference. [F6]

Note: `del x` **does not delete the object**, it deletes the **name** `x`. The object is only destroyed if its count reaches 0. [F2]

3.3 Advantages and the problem

✓ Frees memory **as soon as** it is no longer used (deterministic, no long pauses).

✗ **It does not detect cycles:** if two objects point to each other, their count never reaches 0. [F2]

```
a = []
print(sys.getrefcount(a) - 1) # 1 (a)
a.append(a) # the list contains itself
```

```
print(sys.getrefcount(a) - 1) # 2 (a + the reference inside itself)
del a                        # refcount goes from 2 to 1 -> never reaches 0 on its own
```

```
1
2
```

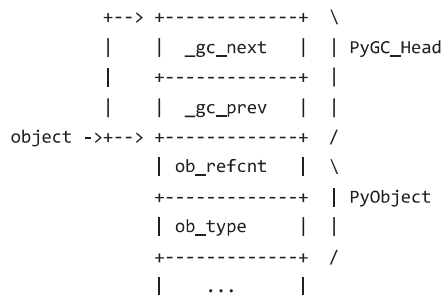
That is what the next layer is for.

4. The cyclic garbage collector (`gc`)

4.1 Which objects does it track?

Only **containers**: objects that can hold references to other objects (lists, dictionaries, classes, instances, etc.). An `int` or a `str` cannot form cycles, so the GC ignores them. **[F2]**

Tracked objects have an extra header, `PyGC_Head`, located **before** the `PyObject`, with two pointers (`_gc_next`, `_gc_prev`) that link them into doubly linked lists. **[F2]**



*(Diagram adapted from **[F2]**)*

```
import gc

print(gc.is_tracked(42))      # False → an int cannot form cycles
print(gc.is_tracked("hello")) # False → neither can a string
print(gc.is_tracked([]))     # True  → a list can
```

```
False
False
True
```

4.2 Algorithm for finding unreachable cycles

Summary of the algorithm described in **[F2]**:

1. For each tracked container, its refcount is copied into a temporary field `gc_ref`.
2. All containers are traversed and, for every **internal** reference (from one container in the group to another), 1 is subtracted from the target's `gc_ref`.
3. Whatever remains with `gc_ref > 0` has references coming from **outside** the group (variables, the stack, etc.) → those objects are **reachable**.
4. Everything those reachable objects point to is also marked as reachable.
5. Whatever is left with `gc_ref == 0` and was not reached is **cyclic garbage**: finalizers (`__del__`) are run, weak references are cleared, and the objects are freed.

```
import gc

class Node:
    def __init__(self):
        self.other = None

a, b = Node(), Node()
a.other, b.other = b, a # cycle a <-> b
del a, b                # each one's refcount = 1 (they point to each other)

print(gc.collect())     # the GC finds and frees the cycle (> 0 objects)
```

```
8
```

4.3 Generations: "most objects die young"

Traversing **all** objects on every collection would be expensive. CPython relies on the **generational hypothesis**: most objects die shortly after being created. **[F2]**

- **Up to Python 3.13**: three generations (0, 1, 2). New objects enter generation 0; if they survive a collection they move to the next one. Generation 0 is checked often and generation 2 rarely. **[F2] [F7]**
- **Python 3.14**: the GC becomes **incremental** with only two generations (**young** and **old**); the old generation is processed in small steps to reduce pauses. **[F2] [F8]**

Collection is triggered automatically based on **thresholds** (number of container allocations minus deallocations), which can be inspected and modified. **[F7]**

```
import gc

print(gc.get_threshold()) # per-generation thresholds (values vary by version)
print(gc.get_count())    # current counters
gc.disable()             # disables the automatic GC (reference counting keeps working)
gc.enable()

(2000, 10, 10)
(165, 0, 0)
```

4.4 Finalizers (`__del__`)

Since Python 3.4 (PEP 442) the GC can collect cycles even if their objects define `__del__`; before that they got "stuck" in `gc.garbage`. Each finalizer runs **at most once** per object. **[F2] [F11]**

4.5 Free-threaded build

In the build without the GIL, the GC **stops the other threads** ("stop-the-world") while collecting, and uses a 1-byte field `ob_gc_bits` in every object to mark its state. In addition, reference counting becomes **biased** (*biased reference counting*): the thread that owns the object modifies a local counter without atomic operations, while other threads use a shared counter. **[F2] [F10]**

5. Optimizations: objects that get reused

5.1 Cached small integers

CPython pre-creates the integers from **-5 to 256**; every time one of those values is used, the same object is reused. **[F12]**

```
a = 256; b = 256
print(a is b) # True (same cached object)

a = 257; b = 257
print(a is b) # False (two different objects)

True
False
```

This is why values should be compared with `==` and not with `is`.

5.2 String interning

Some strings are **interned**: a single copy is stored and every occurrence points to it. This speeds up dictionary lookups (variable names, attributes, keys). CPython automatically interns identifiers and single-character latin-1 strings, and interning can be forced with `sys.intern()`. **[F3] [F6]**

```
import sys

# Strings built at runtime → different objects
a = "".join(["hel", "lo"])
b = "".join(["hel", "lo"])
print(a == b) # True → same value
print(a is b) # False → two different objects on the heap

# Interned → the same copy
a = sys.intern(a)
b = sys.intern(b)
print(a is b) # True → both now point to the object in the table

True
False
True
```

5.3 Immortal objects (Python 3.12+)

Objects such as `None`, `True`, `False` and the small integers are **immortal**: their refcount is a special value that never changes, so they are never freed. This avoids writes to shared memory and makes multithreading easier. [F9]

```
import sys

print(sys.getrefcount(None)) # huge, constant number in 3.12+

3221225511
```

6. Tools for observing memory

Tool	What it is for	Source
<code>sys.getrefcount(obj)</code>	Reference count (+1)	[F6]
<code>sys.getsizeof(obj)</code>	Size in bytes of the object (not counting the objects it points to)	[F6]
<code>id(obj)</code>	Identity of the object (in CPython, its memory address)	[F13]
<code>gc.collect()</code> , <code>gc.get_objects()</code> , <code>gc.get_referrers()</code>	Force a collection and inspect tracked objects	[F7]
<code>gc.freeze()</code>	Move objects to a permanent generation (useful before <code>fork()</code>)	[F7]
<code>weakref</code>	References that do not increase the refcount (caches, observers)	[F14]
<code>tracemalloc</code>	Trace where memory was allocated and detect leaks	[F15]

```
import sys
import tracemalloc

print(sys.getsizeof([])) # empty list
print(sys.getsizeof([1])) # list with one pointer (to an int)
print(sys.getsizeof(["I have the same size as a pointer"])) # same size as a pointer
print(sys.getsizeof([1, 2, 3, 4, 5])) # grows with the pointers, not with the ints

tracemalloc.start()
data = [str(i) for i in range(100_000)]
current, peak = tracemalloc.get_traced_memory()
print(f"current={current/1024:.0f} KiB, peak={peak/1024:.0f} KiB")
tracemalloc.stop()

56
64
64
104
current=5265 KiB, peak=5285 KiB
```

7. Summary

1. **Everything is an object** in a private heap; variables are **references**. [F4]
2. Memory for small objects (≤ 512 B) is managed by **pymalloc** with arenas $\rightarrow$ pools $\rightarrow$ blocks. [F5]
3. **Reference counting** frees an object as soon as its count reaches 0. [F2]
4. The generational **cyclic GC** finds and frees cycles that reference counting cannot. [F2] [F7]
5. CPython **reuses** common objects: small integers, interned strings, immortal objects, free lists. [F3] [F9] [F12]

Sources

Id	Resource
F1	Python Developer's Guide — CPython's internals: https://devguide.python.org/internals/
F2	CPython <code>InternalDocs/garbage_collector.md</code> — Garbage collector design: https://github.com/python/cpython/blob/main/InternalDocs/garbage_collector.md
F3	CPython <code>InternalDocs/string_interning.md</code> : https://github.com/python/cpython/blob/main/InternalDocs/string_interning.md
F4	Python docs — Memory Management (C API), Overview: https://docs.python.org/3/c-api/memory.html#overview
F5	Python docs — Memory Management (C API), sections <i>The pymalloc allocator</i> , <i>Allocator Domains</i> , <i>mimalloc</i> : https://docs.python.org/3/c-api/memory.html#the-pymalloc-alloc
F6	Python docs — <code>sys</code> module (<code>getrefcount</code> , <code>getsizeof</code> , <code>intern</code>): https://docs.python.org/3/library/sys.html
F7	Python docs — <code>gc</code> module: https://docs.python.org/3/library/gc.html
F8	<i>What's New in Python 3.14</i> — Incremental garbage collection: https://docs.python.org/3/whatsnew/3.14.html#whatsnew314-incremental-gc
F9	PEP 683 — <i>Immortal Objects, Using a Fixed Refcount</i> : https://peps.python.org/pep-0683/
F10	PEP 703 — <i>Making the Global Interpreter Lock Optional in CPython</i> : https://peps.python.org/pep-0703/
F11	PEP 442 — <i>Safe object finalization</i> : https://peps.python.org/pep-0442/
F12	Python docs — <i>Integer Objects</i> (<code>PyLong_FromLong</code>), cache from -5 to 256: https://docs.python.org/3/c-api/long.html
F13	Python docs — <code>id()</code> function: https://docs.python.org/3/library/functions.html#id
F14	Python docs — <code>weakref</code> module: https://docs.python.org/3/library/weakref.html
F15	Python docs — <code>tracemalloc</code> module: https://docs.python.org/3/library/tracemalloc.html